Programming Languages Concepts

LAB #2 / PART 1

Basic Operators in Programming Language

Chosen implementation language: Ruby

Student name: Igors

Student surname: Oļeiņikovs

St. code: 93642

Group: 4501BTA

Lecturer: Karina Kostjkina

RIGA

2026

Table of Contents

1. Purpose & Task
2. Program Code & Results
3. Conclusions
4. Literature List

1. Purpose & Task

The purpose of this laboratory work is to study basic arithmetic and conversion operators in a programming language and to compare the chosen language with other programming languages known by the student. Ruby was selected for implementation because it has concise syntax, expressive built-in numeric operations, and readable string interpolation.

Tasks completed in this report:

1. Read two integers and output their product and integer quotient.
2. Read three real numbers, compute their sum, and output the result with two decimal places.
3. Calculate the volume of a room in cubic metres and cubic centimetres.
4. Ask the user for age in years and output the age on the next birthday.
5. Express a number of days in weeks and days.
6. Express a number of seconds in hours, minutes, and seconds.
7. Convert kilometres per hour to miles per hour using 1 km = 0.62137 mile.

2. Program Code & Results

2.1. Task 1

Write a program which reads two integers and displays their product and the integer quotient when the first is divided by the second.

Program Code:

```
#!/usr/bin/env ruby

def read_integer(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Integer(input, exception: false)
    return value unless value.nil?

    puts 'Please enter a valid integer.'
  end
end

first_number = read_integer('Enter the first integer: ')
second_number = read_integer('Enter the second integer: ')

product = first_number * second_number

puts "Product: #{product}"

if second_number.zero?
  puts 'Integer quotient: undefined (division by zero).'
else
  quotient = first_number.div(second_number)
  puts "Integer quotient (DIV): #{quotient}"
end
```

Task 1 Result

task1_product_and_integer_quotient.rb

```
$ ruby task1_product_and_integer_quotient.rb
Enter the first integer: 12
Enter the second integer: 5
Product: 60
Integer quotient (DIV): 2
```

Fig. 1 Task 1 result

2.2. Task 2

Write a program which accepts three real numbers, computes their sum, and outputs the result. Amend the program so that the result is output to 2 decimal places.

Program Code:

```
#!/usr/bin/env ruby

def read_float(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Float(input, exception: false)
    return value unless value.nil?

    puts 'Please enter a valid real number.'
  end
end

first_number = read_float('Enter the first real number: ')
second_number = read_float('Enter the second real number: ')
third_number = read_float('Enter the third real number: ')

sum = first_number + second_number + third_number

puts "Sum: #{sum}"
puts format('Sum rounded to 2 decimal places: %.2f', sum)
```

Task 2 Result

task2_sum_of_reals.rb

```
$ ruby task2_sum_of_reals.rb
Enter the first real number: 1.25
Enter the second real number: 2.5
Enter the third real number: 3.75
Sum: 7.5
Sum rounded to 2 decimal places: 7.50
```

Fig. 2 Task 2 result

2.3. Task 3

Write a program that takes in the width, length, and height of a room in metres and returns the volume of the room in cubic metres and cubic centimetres.

Program Code:

```
#!/usr/bin/env ruby

def read_float(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Float(input, exception: false)
    if !value.nil? && value >= 0
      return value
    end

    puts 'Please enter a valid non-negative number.'
  end
end

width = read_float('Enter the room width in metres: ')
length = read_float('Enter the room length in metres: ')
height = read_float('Enter the room height in metres: ')

volume_cubic_metres = width * length * height
volume_cubic_centimetres = volume_cubic_metres * 1_000_000

puts format('Volume: %.3f cubic metres', volume_cubic_metres)
puts format('Volume: %.0f cubic centimetres', volume_cubic_centimetres)
```

Task 3 Result

task3_room_volume.rb

```
$ ruby task3_room_volume.rb
Enter the room width in metres: 4.5
Enter the room length in metres: 3.2
Enter the room height in metres: 2.8
Volume: 40.320 cubic metres
Volume: 40320000 cubic centimetres
```

Fig. 3 Task 3 result

2. Program Code & Results (continued)

2.4. Task 4

Ask the user for age in years and then tell the user how old they are going to be on the next birthday.

Program Code:

```
#!/usr/bin/env ruby

def read_integer(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Integer(input, exception: false)
    return value if !value.nil? && value >= 0

    puts 'Please enter a valid age in whole years.'
  end
end

age = read_integer('Enter your age in years: ')

puts "On your next birthday you will be #{age + 1} years old."
```

Task 4 Result

task4_next_birthday.rb

```
$ ruby task4_next_birthday.rb
Enter your age in years: 19
On your next birthday you will be 20 years old.
```

Fig. 4 Task 4 result

2.5. Task 5

Write a program to express a number of days in weeks and days.

Program Code:

```
#!/usr/bin/env ruby

def read_integer(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Integer(input, exception: false)
    return value if !value.nil? && value >= 0

    puts 'Please enter a valid non-negative integer.'
  end
end

total_days = read_integer('Enter the number of days: ')
weeks, days = total_days.divmod(7)

puts "#{total_days} day(s) = #{weeks} week(s) and #{days} day(s)."
```

Task 5 Result

task5_days_to_weeks.rb

```
$ ruby task5_days_to_weeks.rb
Enter the number of days: 17
17 day(s) = 2 week(s) and 3 day(s).
```

Fig. 5 Task 5 result

2.6. Task 6

Write a program to express a given number of seconds in terms of hours, minutes, and seconds.

Program Code:

```
#!/usr/bin/env ruby

def read_integer(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Integer(input, exception: false)
    return value if !value.nil? && value >= 0

    puts 'Please enter a valid non-negative integer.'
  end
end

total_seconds = read_integer('Enter the number of seconds: ')
hours, remaining_seconds = total_seconds.divmod(3600)
minutes, seconds = remaining_seconds.divmod(60)

puts "#{total_seconds} second(s) = #{hours} hour(s), #{minutes}
minute(s), #{seconds} second(s)."
```

Task 6 Result

task6_seconds_to_time.rb

```
$ ruby task6_seconds_to_time.rb
Enter the number of seconds: 3665
3665 second(s) = 1 hour(s), 1 minute(s), 5 second(s).
```

Fig. 6 Task 6 result

2.7. Task 7

Write a program to convert km/h into mph using 1 km = 0.62137 mile.

Program Code:

```ruby
#!/usr/bin/env ruby

def read_float(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Float(input, exception: false)
    return value if !value.nil? && value >= 0

    puts 'Please enter a valid non-negative speed.'
  end
end

speed_kmph = read_float('Enter the speed in km/h: ')
speed_mph = speed_kmph * 0.62137

puts format('%.2f km/h = %.2f mph', speed_kmph, speed_mph)
```

Task 7 Result

task7_kmph_to_mph.rb

```
$ ruby task7_kmph_to_mph.rb
Enter the speed in km/h: 90
90.00 km/h = 55.92 mph
```

Fig. 7 Task 7 result

3. Conclusions

All seven tasks were successfully implemented in Ruby. The programs use the basic arithmetic operations of the language, integer division with `div`, quotient and remainder extraction with `divmod`, floating-point calculations, and formatted output through `format`. Sample runs confirm that the expected results were achieved.

Ruby is very convenient for this kind of assignment because its syntax is concise and easy to read. Compared with C++ or Java, Ruby requires much less boilerplate code: no explicit type declarations, no semicolons, and no class wrapper for simple console programs. Compared with Python, Ruby is similarly readable, but it provides some expressive numeric helpers such as `div` and `divmod` that map naturally to the task requirements.

The main advantage of Ruby in this laboratory work is readability and speed of development. The main disadvantage is that Ruby is dynamically typed, so some mistakes are detected only at runtime rather than at compile time. For short educational programs, however, Ruby is a strong choice because it lets the programmer focus on the task logic instead of language ceremony.

4. Literature List

1. Programming Languages Concepts course materials, Lab 2. Basic Operators in PL/Part 1.
2. Ruby Documentation. Core classes `Integer`, `Float`, and `Numeric`. Available at: https://ruby-doc.org/
3. Ruby Documentation. Method `Integer#div` and `Integer#divmod`. Available at: https://ruby-doc.org/core/
4. Metric conversion constant used in task 7: 1 kilometre = 0.62137 mile.